Lexical and Semantic Retrieval for News Recommendation

Problem and Scope

The goal is to compare lexical and semantic candidate generation for MIND and EB-NeRD. The pipeline therefore has to do more than retrieve articles: it also needs typed data tables, temporal availability checks, ranking and beyond-accuracy metrics, scaling measurements, and valid Codabench submissions. I use the title and abstract as the article text for both datasets. The offline experiments use MIND-small and EB-NeRD-demo; the large, unlabelled bundles are used for the final leaderboard predictions.

Data and Experimental Protocol

The raw files are converted into validated Parquet tables for articles, histories, impressions, candidates, and retrieval results. Identifiers are namespaced and time-stamps are normalized. EB-NeRD provides publication times, so those determine whether an article is available. MIND does not provide the same field; there I use the earliest observation across histories, exposures, and clicks as the availability boundary.

The split is temporal rather than random. Earlier training impressions provide the statistics, the latest complete training day is validation, and the labelled validation package is the offline test period. At serving time, retrieval can only use articles that were already available at the impression timestamp. This prevents both future articles and future clicks from entering the candidate set.

The final verified run is `assignment-final-all-20260826T145844Z-9255ee8`, at revision `9255ee8`. Download, preparation, retrieval, evaluation, benchmarking, plotting, and submission are separate stages. A cached stage is reused only when its code, configuration, inputs, and recorded outputs still match. Large scoring jobs also write checkpointed chunks, which makes an interrupted run restartable without mixing results from different revisions.

Retrieval Methods

BM25

BM25 indexes the normalized title and abstract. Tokenization lowercases text, removes stopwords, preserves Danish diacritics, and does not stem. I use $k_1 = 1.2$ and $b = 0.75$. A user query is formed by concatenating the most recent eligible clicked articles. When there is no history, the system falls back to training-only popularity, with article IDs resolving ties deterministically.

Dense retrieval

For MIND I use `BAAI/bge-m3`. EB-NeRD is Danish, so I use the supplied multilingual BERT vectors rather than applying an English encoder to it. User vectors are L2-normalized averages of the eligible recent article vectors. Exact FAISS inner-product search is the main dense retriever; HNSW is included in the scaling comparison. Validation selects among history lengths 5, 10, 20, and all independently for each dataset. Dense retrieval can connect related wording that BM25 misses, but its result depends on the encoder and the language of the data. Ties and output order are deterministic.

Evaluation

I report AUC, MRR, nDCG@5, and nDCG@10 for ranking. Full-corpus retrieval is evaluated with Recall@50, Recall@100, and Recall@200. For beyond-accuracy, diversity is the mean pairwise cosine distance, novelty is Laplace-smoothed self-information from training clicks, and coverage is the number of unique recommendations divided by the exposed candidates.

Users are split into cold and warm groups at the median untruncated history length. Head articles are the smallest set covering 80% of training clicks. The 95% intervals use 1,000 user-clustered bootstrap draws with seed 146; coverage resamples recommendations and exposures together. Undefined cases are excluded from the estimate and counted. Recall asks whether the relevant article entered the candidate set, while AUC, MRR, and nDCG ask how the supplied candidates were ordered, so the two kinds of measurement answer different questions.

Metric	MIND BM25	MIND BGE	EB-NeRD BM25	EB-NeRD BGE
AUC	.5583 [.5525, .5640]	.5944 [.5884, .6007]	.5054 [.4989, .5128]	.4905 [.4835, .4978]
MRR	.2947 [.2886, .3007]	.3301 [.3237, .3365]	.3194 [.3129, .3262]	.3174 [.3112, .3242]
nDCG@5	.2706 [.2639, .2768]	.3058 [.2989, .3124]	.3502 [.3426, .3587]	.3456 [.3378, .3540]
nDCG@10	.3319 [.3255, .3377]	.3658 [.3597, .3721]	.4362 [.4295, .4428]	.4298 [.4230, .4373]

Table 1: Overall offline ranking results. Smaller gray text gives the 95% confidence interval.

Metric	MIND BM25	MIND BGE	EB-NeRD BM25	EB-NeRD BGE
R@50	.0072	.0129	.0122	.0096
R@100	.0127	.0219	.0207	.0196
R@200	.0195	.0385	.0353	.0366
Div@10	.5070	.4439	.0483	.0068
Nov@10	16.9248	16.8430	13.8456	14.0798
Cov@10	.1926	.1790	.3427	.1317

Table 2: Overall recall and beyond-accuracy results.

Results and Analysis

On MIND-small, BGE improves every accuracy metric. AUC rises from 0.5583 to 0.5944, and Recall@200 rises from 0.0195 to 0.0385. BM25 still has higher diversity and coverage. In other words, improving relevance does not automatically make the recommendations broader.

On EB-NeRD-demo, BM25 is better on AUC, MRR, both nDCG values, and coverage. BGE is close on Recall@200, but its diversity and coverage are much lower. The likely explanation is a mismatch between the representation and this Danish dataset, but these results should not be read as a general rule that lexical retrieval is always better than semantic retrieval.

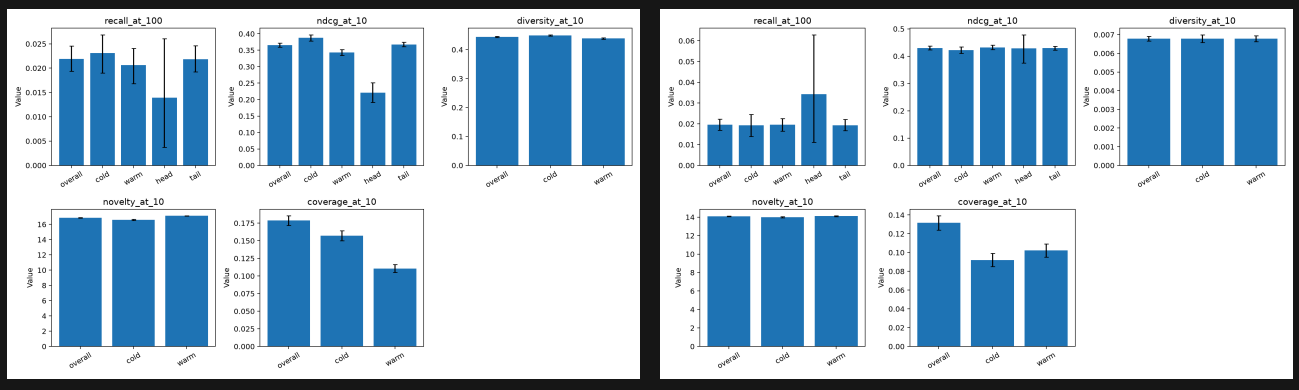

Figure 1: Cold/warm and head/tail slices for the dense systems.

The generated plots include PCA, sampled t-SNE, recall, scaling, and memory views. The slices suggest that cold users depend more on the popularity fallback, while tail items have weaker lexical and click evidence. Dense profiles can also concentrate recommendations around semantically similar articles. These plots are useful diagnostics, but they do not establish causation.

Scaling and the 10x Question

For scaling, I use the first 1,000 test impressions at 25%, 50%, and 100% workload, with three repetitions. Each run records time, throughput, sampled peak RSS and CUDA memory, index size and parameters, workload and ranking hashes, and fixed predictions. The 10x column is only a linear extrapolation; it is not a measurement on the hidden test set.

MIND	Time	Imp./s	RSS	Index	10x
BM25	22.61	44.24	2154	13.5	226.06
Exact	24.29	41.18	2774	255.2	242.86
HNSW	10.04	99.64	2809	271.8	100.36

EB-NeRD	Time	Imp./s	RSS	Index	10x
BM25	3.58	279.72	1486	1.6	35.75
Exact	6.47	154.57	2170	34.5	64.70
HNSW	3.01	332.32	2589	37.6	30.11

Table 3: 100% benchmark measurements. Time and 10x are seconds, throughput is impressions/s, and RSS/index size are MiB.

Here HNSW is faster than exact search, at the cost of a somewhat larger index. At competition scale, the expensive parts are building the index, storing vectors, and scoring millions of candidate lists. The linear 10x estimate can therefore change with the cache, I/O behavior, and candidate distribution. Bounded Parquet batches, checkpointed chunks, and stage validation reduce the chance of an OOM or interrupted job, but they do not make the underlying computation cheap. RSS and persisted index size are separate measurements because a compact index can still require substantial memory while querying.

Codabench Submissions

The MIND archives contain a root-level `prediction.txt`, while the EB-NeRD archives contain `predictions.txt`. The validator checks the archive structure, row order, row count, and the fact that each impression receives a complete ranking beginning at one. I generated two MIND systems, BM25 and BGE, and two EB-NeRD variants using history lengths 10 and 20, from the required large bundles. The two MIND submissions scored 0.5883 and 0.5853. The EB-NeRD screenshot confirms that both archives were submitted, but it does not show their scores.

Reproducibility, Limitations, and Conclusion

Pixi rebuilds the project and records the source and configuration identity, device, stage state, checksums, metrics, bootstrap draws, plots, and submissions. The complete implementation is available at https://gitlab.com/skeleton-hacker-acads/ire_assn_1. The final evidence bundle validates without errors. The tests cover schemas, temporal splits, availability, leakage, deterministic ranking, metrics, slices, bootstrap reproducibility, and end-to-end execution.

There are several limitations. I do not train a lexical-semantic ranker or fine-tune the encoders, the two datasets use different encoders, the session model is limited to recent clicks, and article bodies are not used. BGE is strongest on MIND-small, while BM25 is more reliable on EB-NeRD-demo. At 10x, dense retrieval and serialization are still the main resource risks, even with batching and checkpointing.

Leaderboard Evidence

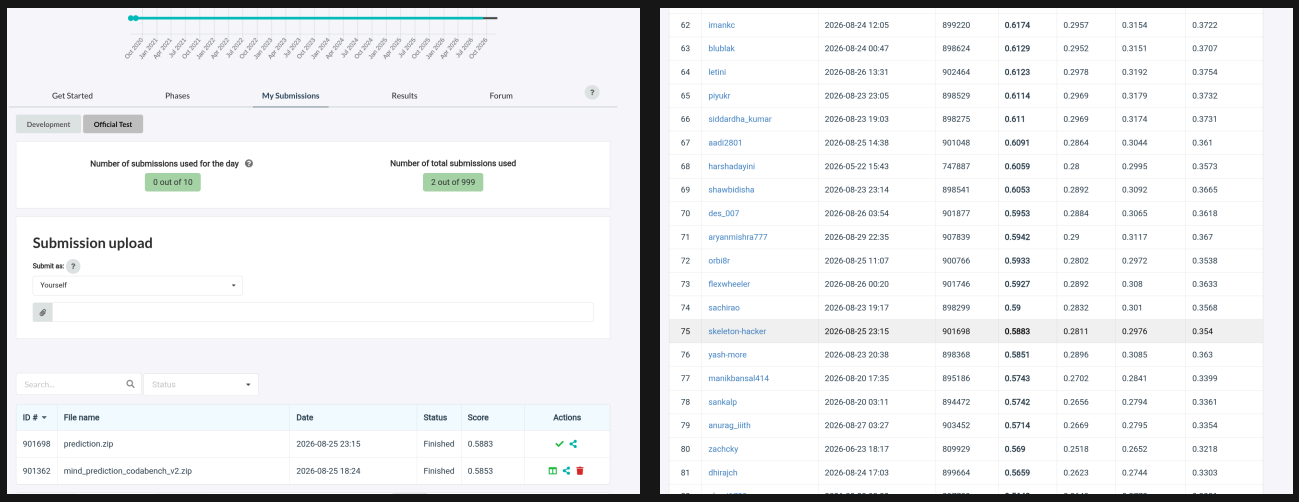

Figure 2: MIND submission scores and the corresponding leaderboard entry.

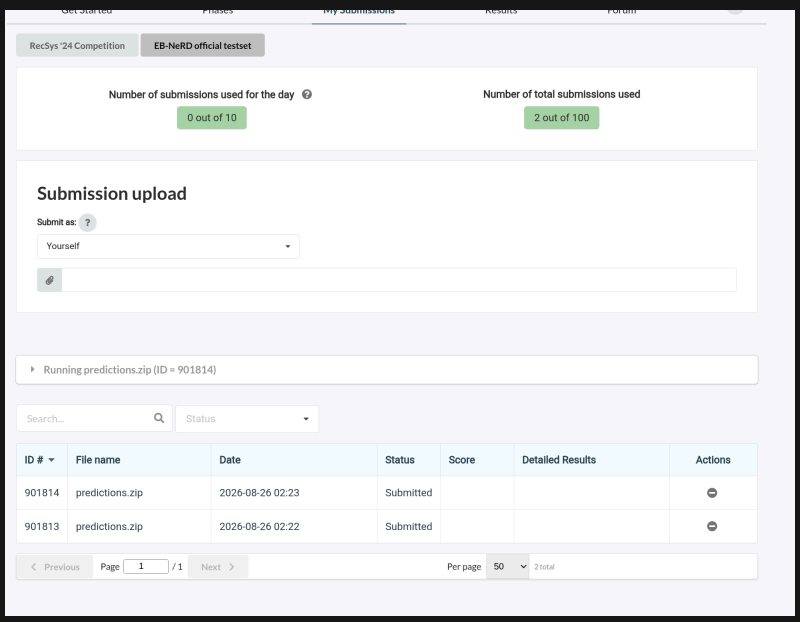

Figure 3: Two EB-NeRD official-test submissions recorded on Codabench.